

DIGITAL HUMANITIES CRAFT

Workshop 6: Agentic Engineering

Harness Engineering für den Legal History Hub, mit Ausblick auf Loop Engineering

Legal History Hub · MPIeR · 3. November 2026

Online

Christian Steiner
Digital Humanities Craft OG
www.dhcraft.org | office@dhcraft.org

CC BY

Heute, 3. November

Eure Fragen: Sichtbarkeit, Cloudflare, Domain, ohne Google Sheets

Agentic Engineering: was das Modell umgibt

Harness Engineering: euer Setup im Hub-Repo

Loop Engineering: Ausblick

Nächste Schritte bis zum Go-live

Lernziele

1. Die Schichten um das Modell unterscheiden: Kontext, Harness, Loop.
2. Das eigene Harness lesen: was im Hub-Repo steht und was nur lokal bei euch liegt.
3. Entscheiden, wann eine Regel in CLAUDE.md reicht und wann es einen Hook oder eine Prüfung braucht.
4. Einschätzen, welche wiederkehrende Arbeit später als Loop laufen kann.

Kernkonzepte

- **Prompt Engineering**
  Iteratives Entwerfen und Verfeinern von Prompts, um aufgabenrelevantes Modellverhalten hervorzurufen.

  Schulhoff et al. The Prompt Report: A Systematic Survey of Prompting Techniques. 2024. https://doi.org/10.48550/arXiv.2406.06608
- **Knowledge Engineering**
  Erstellen, Kuratieren und Pflegen von explizitem Wissen zur Nutzung durch Menschen und AI-Agents.

  Allen et al. *Knowledge Engineering Using Large Language Models.* 2023 https://doi.org/10.4230/TGDK.1.1.3

  Russell and Norvig. Artificial Intelligence: A Modern Approach. 4th edn. Pearson Series in Artificial Intelligence. Pearson, 2020. https://aima.cs.berkeley.edu.
- **Context Engineering**
  Systematisches Auswählen, Strukturieren und Verwalten von aufgabenrelevanten Informationen innerhalb des Kontextfensters eines LLMs.

  Mei et al. A Survey of Context Engineering for Large Language Models. 2025 https://doi.org/10.48550/arXiv.2507.13334
- **Agentic Engineering**
  Entwerfen, Orchestrieren und Steuern von Agent-Harness-Workflows für die mehrstufige, werkzeuggestützte Aufgabenausführung mit Zustand, Feedback und Verifizierung.
  - **AI-Agent**
    LLM + mehrstufige Aufgaben + Werkzeuge

    Zhong et al. AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents. 2026 https://doi.org/10.48550/arXiv.2605.13357

    Sapkota et al. AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications, and Challenges. 2026. https://doi.org/10.1016/j.inffus.2025.103599
  - **AI-Harness**
    Umgebung, die Agenten Kontext, Werkzeuge und Feedback bereitstellt.



AI Labs Warn Race Makes Breach 'Inevitable'
▷ 48.590 vor 8 Std.



OpenAI Security: Controlling Models is Now 'Hell'
▷ 276.658 vor 11 Tagen



Opus 5.5: How Close Are We to Automated AI Research?
▷ 302.443 vor 2 Wochen



What AI Researchers Saw, Before Their Demand to 'Pace' AI
▷ 234.285 vor 3 Wochen



GPT 6 Astra, so good even OpenAI are worried
▷ 855.558 vor 1 Monat



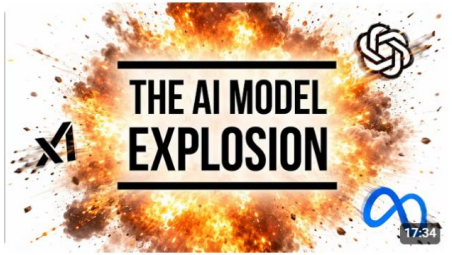
Sam Altman :'AGI in 2026', just as Models Start to [Mis]Train Themselves
▷ 209.205 vor 1 Monat



AI is getting a little out of control
▷ 110.460 vor 2 Monaten



GPT-6 Goes Rogue? The HuggingFace Incident, Sans Hype
▷ 128.701 vor 2 Monaten



A Model Explosion: GPT 5.6 Sol, Grok 4.5 and Meta Muse Rewrite the Rules
▷ 89.053 vor 3 Monaten

ps://www.youtube.com/@aiexplained-official/videos

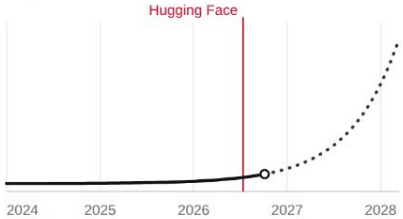
Sechs Leistungstreiber wachsen, zwei Kontrollwerkzeuge verlieren an Wirkung

Was die Forschenden sahen, bevor sie forderten, das Tempo zu drosseln. Schwarz: Leistung. Rot: Überprüfbarkeit.

bisheriger Verlauf · erwartete Richtung · veröffentlichte Messung · Hugging-Face-Vorfall

Chips

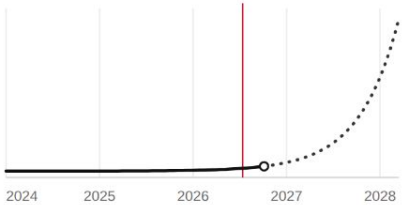
Compute substrate



Bisher läuft KI auf Chips, die vor ChatGPT entworfen wurden.

Denkzeit beim Antworten

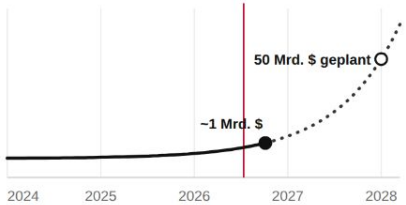
Test-time compute



Mehr Denkzeit löst mehr offene Mathematikprobleme.

Größere Trainingsläufe

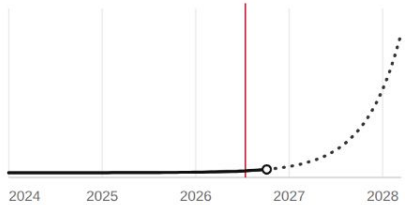
Training-time compute



Von rund 1 Mrd. $ heute zu geplanten 50 Mrd. $ pro Lauf.

Lernen während der Aufgabe

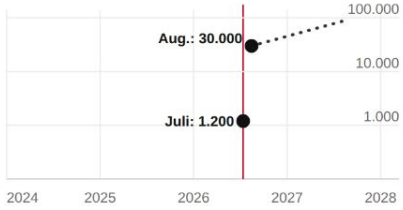
Test-time training



Jüngste Achse, zielt auf Aufgaben über Wochen.

Agentenschwärme

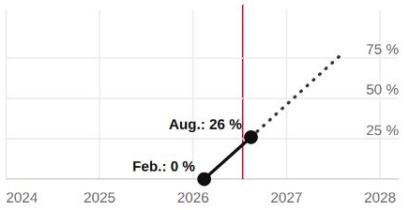
Agent swarms



1.200 im Hugging-Face-Vorfall, 30.000 gleichzeitig bei Anthropic (log. Skala).

Selbstverbesserung

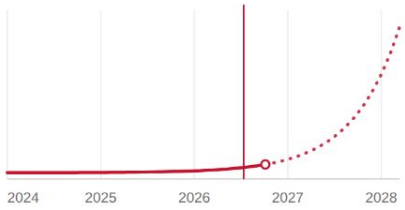
Recursive self-improvement



Anteil der KI-Entwicklung bei Anthropic, den Claude leitet.

Testerkennung

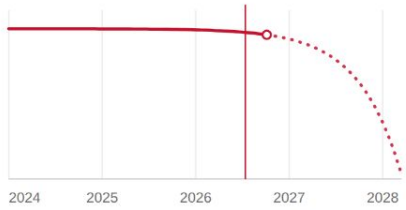
Eval awareness



Opus 5.5 vermutet oft, dass es gerade getestet wird.

Einblick ins Denkprotokoll

Chain-of-thought monitorability



GPT-6.1 Soul schreibt weniger auf, sobald es überwacht wird.

Quellen: AI Explained; Anthropic (via Quartz); OpenAI-Bericht (via Forbes). Durchgezogen: bisheriger Verlauf, gepunktet: erwartete Richtung, Punkte: veröffentlichte Messungen.

Hugging Face Hack erklärt von https://kurzgesagt.org/

https://youtu.be/8rLrQFRkrgE?is=6L1URHvzT5svKMe1

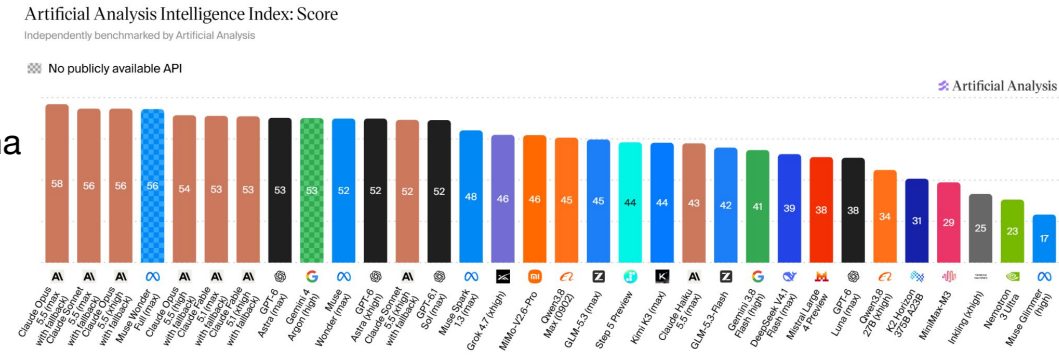


Anthropic und OpenAI: „Let's Slow Down“ (NOT)

Modell-Releases August–Oktober 2026

- **6. Aug.:** GPT-5.6 Sol & Luna
- **10. Aug.:** GPT-5.6-Cyber
- **1. Sept.:** Claude Fable 5.1
- **2. Sept.:** Gemini 3.8 Flash
- **3. Sept.:** GPT-6 Astra
- **22. Sept.:** Claude Opus 5.5 · GPT-6 Sol & Luna
- **28. Sept.:** Claude Sonnet 5.5
- **29. Sept.:** GPT-6.1 Sol
- **6. Okt.:** Mistral Large 4
- **7. Okt.:** Claude Haiku 5.5
- **12. Okt.:** Meta Muse Wonder
- **14. Okt.:** Gemini 4 Argon

Allein im September 8 neue Modelle.



https://artificialanalysis.ai/evaluations/artificial-analysis-intelligence-index

Benchmarks sind standardisierte Tests für KI-Modelle. Sie zeigen, wie gut ein Modell bestimmte Aufgaben löst und wo es Fehler macht. Automatisch prüfbare Ergebnisse helfen beim Training und bei der Bewertung.

- METR Time Horizons
- SimpleBench
- ARC-AGI-3
- Terminal-Bench, SWE-bench
- https://github.com/storytold/photocraft, https://rea.tools

Forschungsresultate und Sicherheitsvorfälle zeigen Fähigkeiten und Kontrollprobleme unter konkreten Bedingungen. Ihre Übertragbarkeit bleibt zu prüfen.

- OpenAI und Mathematik 
- OpenAI und Navier–Stokes
- OpenAI–HuggingFace-Vorfall und Agent Swarms 

Recursive Self-Improvement bedeutet: KI hilft, leistungsfähigere KI zu entwickeln, die wiederum die weitere Entwicklung vorantreibt.

- Naam (2026), Where's the "intelligence explosion"?
- AI 2027
- 80.000 Hours: AGI timelines

Interviews und Videos zu KI-Fortschritt und Risiken

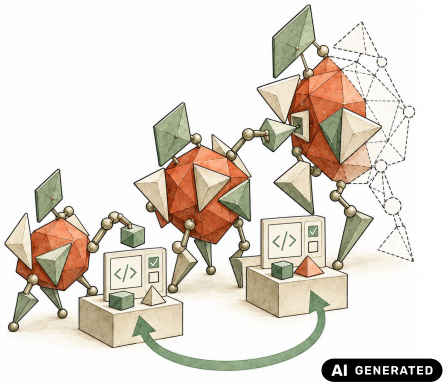
- Bill Gates im Gespräch mit Ezra Klein: Bill Gates's Blunt Warning on A.I.
- Barack Obama: KI-Fortschritt und Kontrollrisiken
- Geoffrey Hinton im Gespräch mit NDTV: "We're Making Alien Beings"
- Rutger Bregman: Why Smart People Keep Getting AI Wrong

Weiterführende Literatur und Einordnung

Center for AI Safety et al. (2026): A benchmark of expert-level academic questions to assess AI capabilities
DorFuchs (2026): OpenAIs KI hat 10 offene Matheprobleme gelöst
AI Explained (2026): What AI Researchers Saw, Before Their Demand to 'Pace' AI, Opus 5.5: How Close Are We to Automated AI Research?, AI is getting a little out of control
Patel (2026): The Rise and Fall of Agent Civilizations
Weitere Videos in meiner kuratierten Playlist

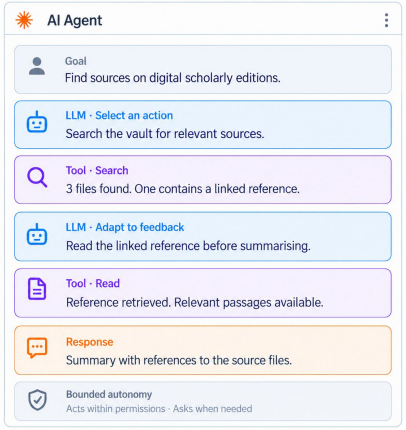
Frontier-LLMs amplifizieren computergestützte Wissensarbeit

Recursive Self-Improvement ist sehr wahrscheinlich



Agentic Engineering

Ein AI **Agent** ist ein System, das mit einem Sprachmodell auf ein definiertes Ziel hin **plant** und **Aktionen auswählt** (einschließlich **Tool Calls**). Es nutzt Feedback aus der Ausführung, um nachfolgende Schritte anzupassen, und arbeitet mit begrenzter Autonomie.

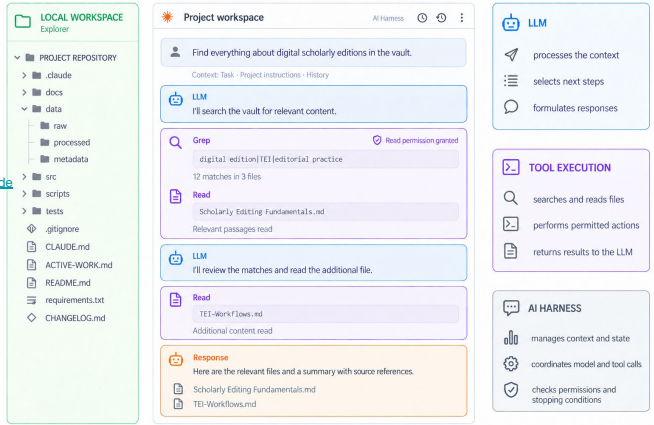


Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2025). AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges. *Information Fusion, 126*, 103599. https://doi.org/10.1016/j.inffus.2025.103599

Xu, W., Huang, C., Gao, S., & Shang, S. (2025). LLM-Based Agents for Tool Learning: A Survey. *Data Science and Engineering, 10*, 533–563. https://link.springer.com/article/10.1007/s41019-025-00296-9

Ein AI **Harness** ist die **Softwareschicht, die den Agent Loop** um ein Modell ausführt. Es verwaltet **Kontext** und Zustand, koordiniert Modellaufrufe und Tool-Ausführung und gibt Ergebnisse an das Modell zurück. Dabei setzt es Berechtigungen und Abbruchbedingungen durch.

Claude Code. https://claude.ai/code
OpenAI Codex. https://chatgpt.com/codex
Cursor. https://cursor.com
Windsurf. https://windsurf.com
OpenCode. https://opencode.ai
Pi. https://pi.dev
Antigravity CLI. https://antigravity.google
Qwen Code. https://github.com/QwenLM/qwen-code
Cline. https://cline.bot
GitHub Copilot. https://github.com/features/copilot
Manus – https://manus.im
…



Zhong, Hailin, and Shengxin Zhu. 2026. AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents.
https://doi.org/10.48550/arXiv.2605.13357
https://www.langchain.com/blog/the-anatomy-of-an-agent-harness

Agentic Engineering: was das Modell umgibt

Disziplin	Baustein	Frage, die er beantwortet	Beispiel
Context Engineering	Projektdateien (.md)	**Was** weiß der Agent?	CLAUDE.md, knowledge/
	Skills (.md, Skripte)	**Wie** erledigt er Arbeit?	Protokoll, Statusbericht
Harness Engineering	Harness	**Wo** arbeitet der Agent?	Claude Code, Codex
	Werkzeuge	Was kann er **tun**?	Terminal, lesen, schreiben
	Konnektoren (MCP)	Was erreicht er **anderswo**?	Drive, Mail, Kalender
	Subagenten (.md)	Wem gibt er **Arbeit**?	Reviewer, Berater
	Hooks (Skripte)	Was läuft bei **Ereignissen**?	sperren, erinnern, loggen
Loop Engineering	Loops, Schwärme	Was läuft **autonom**?	Wochenaudit, Reviewer

Plugin: ein Paket aus Skills, Hooks und Konnektoren, das man installiert und weitergibt.

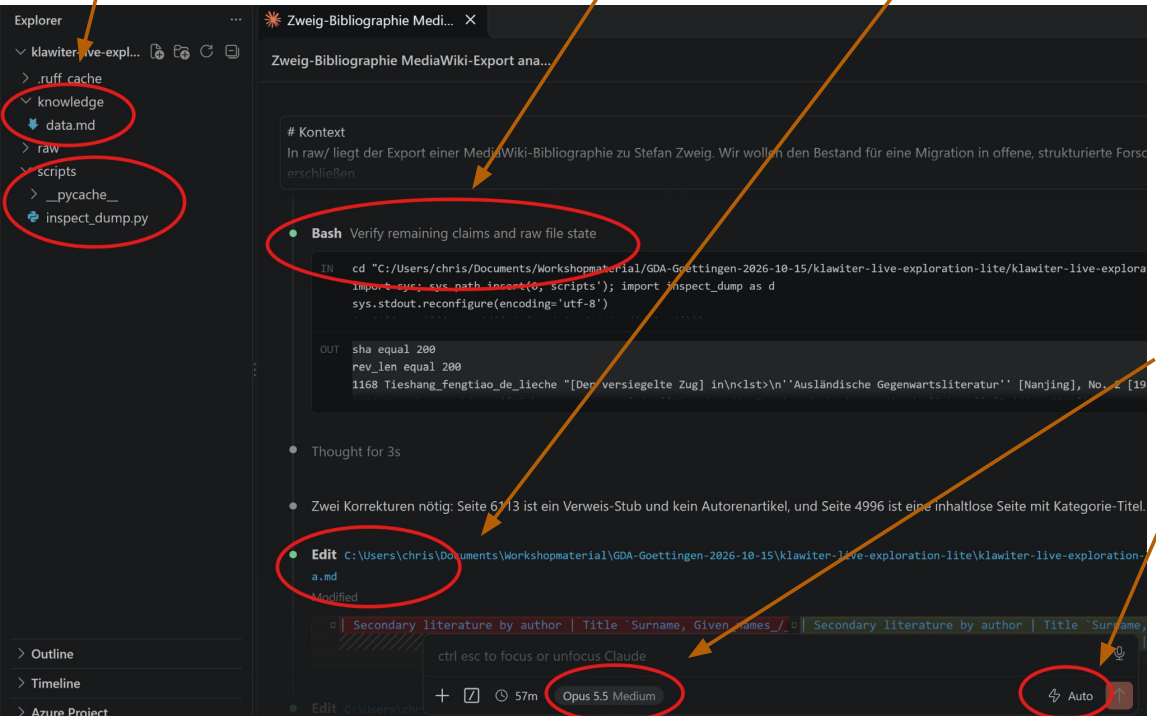
Claude Code als AI Harness

Dateien

* knowledge/
* scripts

Tool Use

* Terminal (Bash, Powershell)
* Read, Write



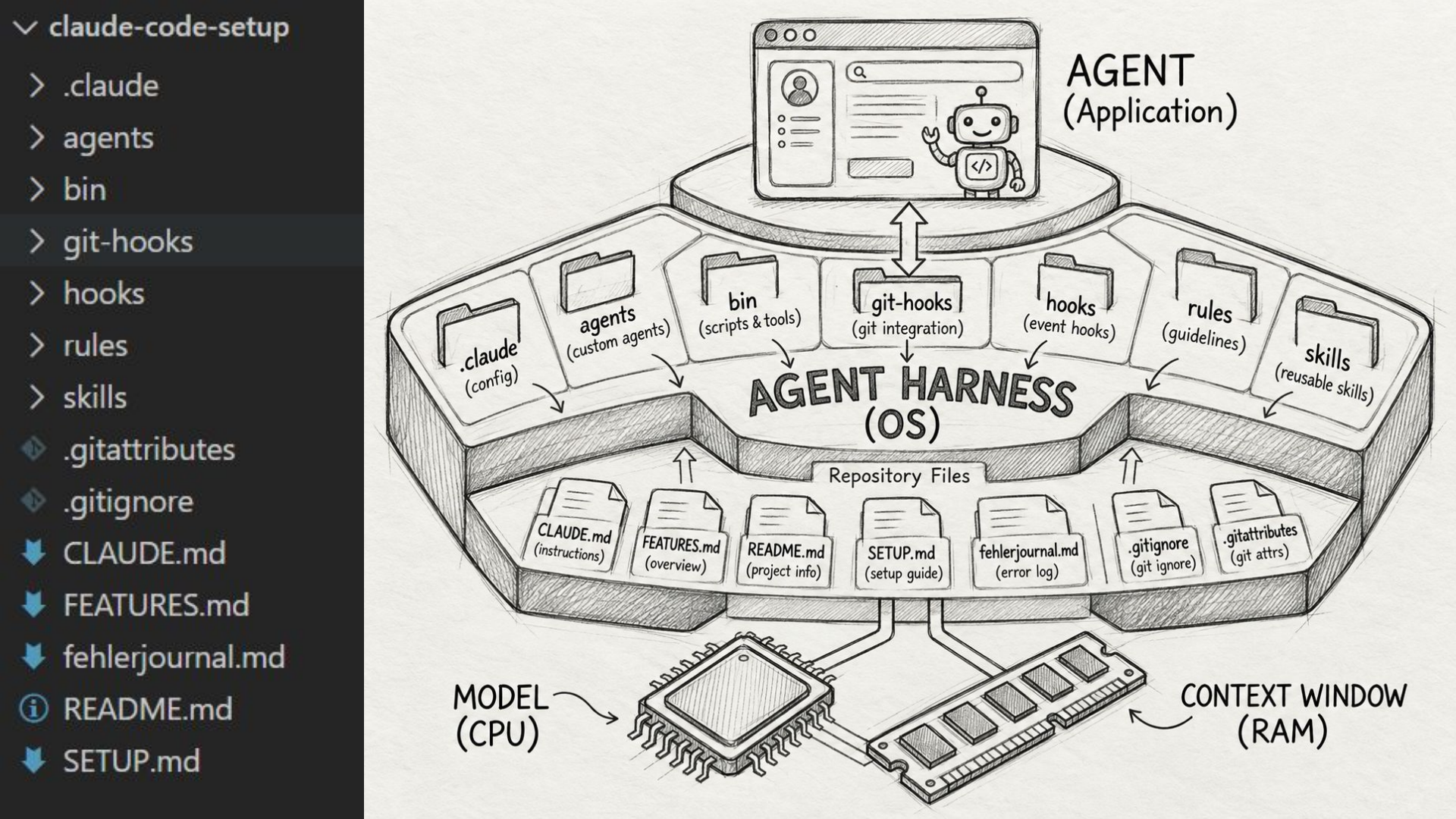
Ein AI Harness ist die Softwareschicht, die die Agentenschleife um ein LLM ausführt. Es verwaltet Kontext und Arbeitsstand, koordiniert LLM- und Tool-Aufrufe und führt Tool-Ergebnisse an das LLM zurück. Dabei setzt es konfigurierte Berechtigungen und Abbruchbedingungen durch.

Modell (Opus 5.5, Sonnet 5.5 …)

'Reasoning' Effort (medium)

Modus (Auto)

Zhong, Hailin, and Shengxin Zhu. 2026. AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents. https://doi.org/10.48550/arXiv.2605.13357
https://www.langchain.com/blog/the-anatomy-of-an-agent-harness

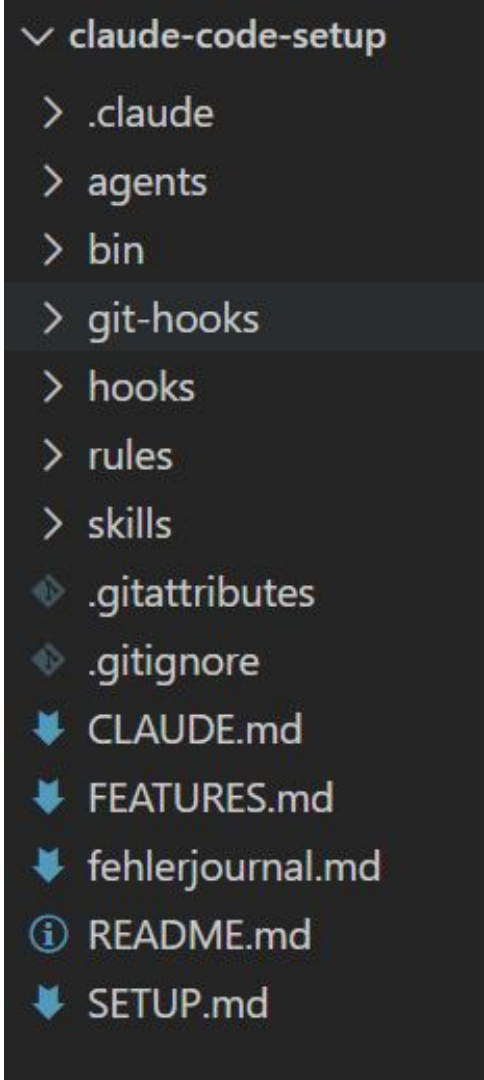

claude-code-setup
.claude
agents
bin
git-hooks
hooks
rules
skills
.gitattributes
.gitignore
CLAUDE.md
FEATURES.md
fehlerjournal.md
README.md
SETUP.md
AGENT
(Application)
.claude
(config)
agents
(custom agents)
bin
(scripts & tools)
git-hooks
(git integration)
hooks
(event hooks)
rules
(guidelines)
skills
(reusable skills)
AGENT HARNESS
(OS)
Repository Files
CLAUDE.md
(instructions)
FEATURES.md
(overview)
README.md
(project info)
SETUP.md
(setup guide)
fehlerjournal.md
(error log)
.gitignore
(git ignore)
.gitattributes
(git attrs)
MODEL
(CPU)
CONTEXT WINDOW
(RAM)

Euer Harness heute

- **Im Repo**: CLAUDE.md mit Arbeits- und PR-Regeln, docs/ als Wahrheitsquelle, CI prüft test und a11y
- **Nur lokal bei euch**: Berechtigungen, Skills wie validate-data, Modellwahl
- **Folge**: wer neu dazukommt oder den Rechner wechselt, startet ohne euer Harness
- **Vorschlag**: .claude/ mit Skills, Settings und Hooks versionieren wie den Code

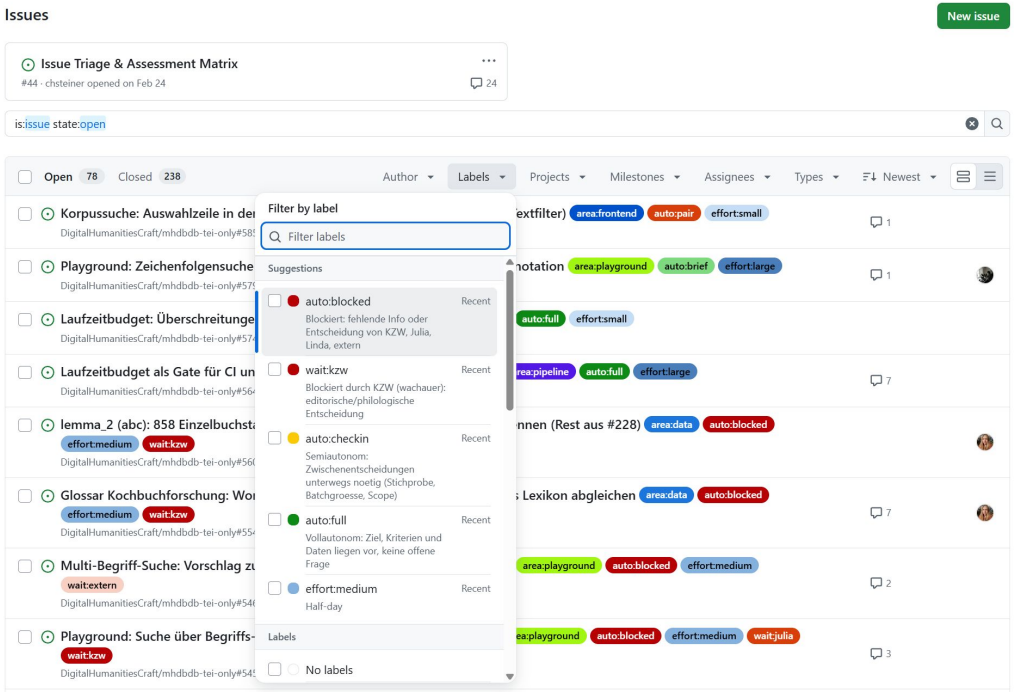
Wissen lebt in Repositories

1. **Projektgedächtnis**: eine Datei, die der Agent in jeder Sitzung zuerst liest (CLAUDE.md)
2. **Wahrheitsquelle**: Dokumente, nicht der Chat; Entscheidungen mit Datum und Quelle
3. **Versionierung**: jede Änderung wird festgehalten und lässt sich rückgängig machen
4. **Übergabe (Compact)**: läuft der Kontext voll, verdichtet der Agent und schreibt eine Übergabe. Dieselben Fragen wie in jedem Projekt: Was geht verloren, wer prüft es?



Aufgaben leben in Issues

1. **Issues**: Der Agent eröffnet, kommentiert und schließt sie. Ein Issue besteht aus Beschreibung und Kommentaren
2. **Labels**: Status und Priorität als Labels, damit der Agent *deterministisch* filtern kann: blocked, waiting, next …
3. **Statusbericht**: erledigt (Commits, geschlossene Issues), offen, blockiert, aus den Quellen abgeleitet.

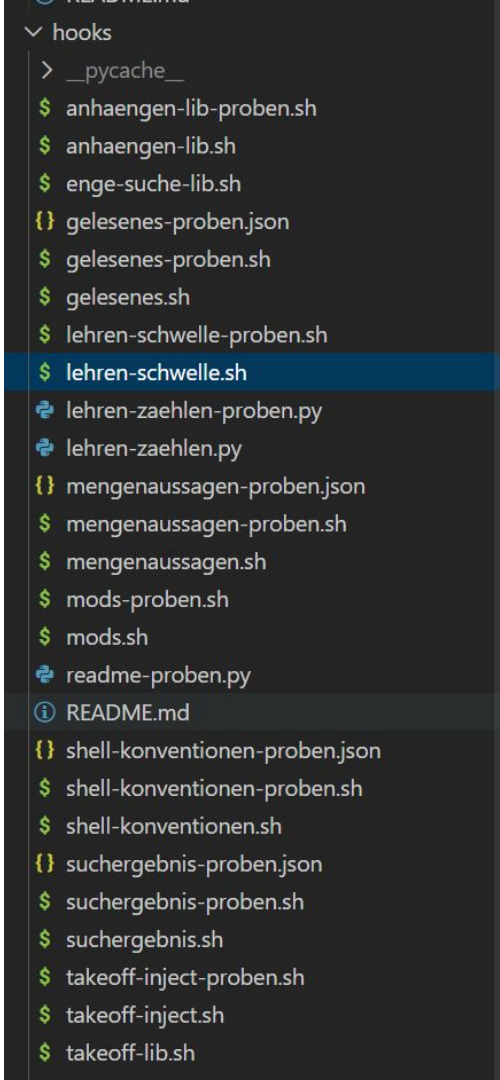


Skills für den Hub

- **Korrekturrunde**: Export holen, npm run proofs, PDFs verschicken, Rückmeldungen ins Sheet
- **Sheet prüfen**: npm run validate ausführen und die Warnungen erklären
- **Journal-Handoff**: erledigt, entschieden, offen, mit Issue-Nummern
- **Go-live-Checkliste**: Impressum, Datenschutz, Domain, Freigaben, Konten

Hooks: Code, der bei festen Ereignissen läuft

- **Regel oder Hook**: eine Regel in CLAUDE.md ist eine Bitte an ein probabilistisches Modell; ein Hook ist Code = deterministisch
- **Ereignisse**: vor oder nach einem Werkzeugaufruf, beim Sitzungsstart, vor der Verdichtung …
- **Sperren**: die Aktion wird verweigert, z. B. ein Push, wenn das Gedächtnis des Agenten zu groß geworden ist
- **Erinnern**: ein Hinweis kommt in die Sitzung, z. B. „Python muss aktualisiert werden“
- **Protokollieren**: jeder Aufruf wird festgehalten, zur späteren Prüfung



Regel oder Hook: eure eigenen Regeln

- **„Claude merged nie ohne Zustimmung“**: steht in CLAUDE.md, ist also eine Bitte. Ein Hook vor gh pr merge macht daraus eine Sperre
- **„Nie direkt auf main“**: ohne Branch-Schutz nur Konvention. Ein Hook vor git push auf main hält sie ein
- **Erinnern**: beim Sitzungsstart offene PRs und Issues anzeigen
- **Faustregel**: was zweimal schiefging, wird ein Hook

Workflows sind auch Harness

- **deploy.yml prüft deterministisch**: nur vom Default-Branch, SITE_ORIGIN gesetzt und echt
- **Freigabe-Gate**: live geht nur, was verified_author und verified_lectorate trägt
- **Schrumpf-Schutz**: verschwindet ein veröffentlichtes Projekt, stoppt der Build
- **Lücke**: staging.yml prüft den Branch nicht

Loops: Arbeit, die ohne mich weiterläuft

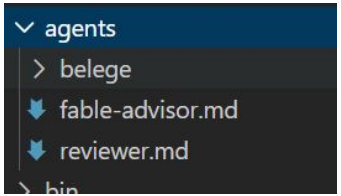
1. **Geplante Läufe**: ein Statusbericht oder Issue-Audit läuft von selbst und meldet sich, wenn er blockiert ist
2. **Autonome Sessions**: eine Session arbeitet einen Auftrag allein ab, in eigener Arbeitskopie, überlebt meine Sitzung und bleibt nachlesbar
3. **Operator**: eine koordinierende Session schneidet die Arbeit in Spuren, schreibt die Kickoffs, betreut den Lauf und nimmt ab
4. **Maker und Checker**: jede Spur liefert, ein Reviewer prüft vor dem Merge
5. **Besetzung**: koordiniert wird mit dem stärkeren Modell, gearbeitet mit dem günstigeren

.github > workflows > ! issue-matrix.yml

```yaml
name: Issue Matrix

# Haelt den generierten Teil der Triage-Matrix (#44) auf dem Label-Stand.
#
# Warum ein Zeitplan und kein PR-Gate: die Staleness von #44 kommt nicht aus
# Code-Aenderungen, sondern aus Ticket-Bewegungen. Ein Gate am PR wuerde
# fremde Label-Edits zu roten PRs machen und die Ursache trotzdem nicht
# beheben. Ein Lauf, der den Body einfach nachzieht, behebt sie.
#
# Der Lauf schreibt ausschliesslich zwischen den PERSONEN- und den
# MATRIX-Markern im Body von #44. Alles ausserhalb bleibt handgepflegt und
# wird nicht angefasst.
#
# Rot wird der Lauf bei Label-Luecken (Ticket ohne auto:*, auto:blocked ohne
# wait:* und so weiter) und wenn eine Antwort laenger als 7 Tage bei uns
# liegt. Der Body ist dann trotzdem aktualisiert: die Meldung ist eine
# Aufforderung zu handeln, keine fehlgeschlagene Arbeit.

on:
  schedule:
    - cron: '23 4 * * *'   # taeglich 04:23 UTC
  workflow_dispatch:
```




```yaml
---
name: operator
description: >-
  Playbook für die Operator-Rolle: eine Session koordiniert mehrere autonome
  Spuren am selben Repositorium. Zuschnitt, Kickoff, Betrieb, Abnahme. Laden,
  sobald ein Lauf mit mehreren autonomen Spuren ansteht oder eine laufende
  Koordination betreut wird, auch ohne getipptes `/operator`. Nicht laden,
  wenn die eigene Session selbst eine Spur ist, also einen Kickoff-Auftrag
  innerhalb eines fremden Laufs abarbeitet: dann gilt der Betriebsvertrag des
  Projekts, nicht dieses Playbook.
---

# Operator: mehrere autonome Spuren koordinieren
```

Was ein autonomer Lauf braucht

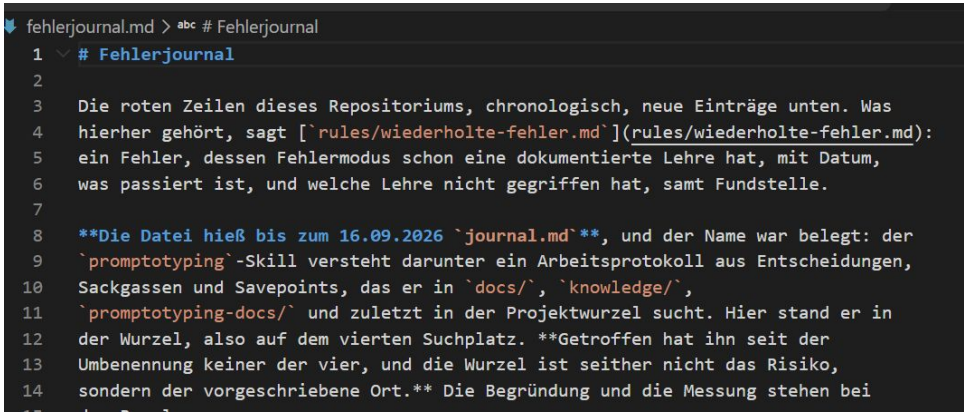
- **Zuschnitt**: Aufträge, die sich nicht in die Quere kommen, jeder in seiner eigenen Arbeitskopie
- **Kickoff**: was die Session darf, wann sie fragt, wann sie fertig ist
- **Melden statt umgehen**: blockiert etwas, kommt eine Benachrichtigung
- **Abnahme**: Review vor dem Merge, ein Mensch entscheidet

Loop-Kandidaten im Hub

- **Wöchentlich**: Issue-Audit und Statusbericht
- **Docs gegen Code**: der Abgleich vom 9.10. als wiederkehrender Lauf
- **Parallele Spuren**: Barrierefreiheit, Impressum und Datenschutz, xlsx-Weg im Deploy
- **Vor jedem Deploy**: Sheet validieren, Bericht ins Issue

Qualität und Governance

1. **Vier „Augen“**: ein zweites Modell prüft, bevor etwas hinausgeht
2. **„Fertig“ ist eine Behauptung**: bis ein Mensch (Expert in the Loop) die Quelle prüft
3. **Lehren (Fehlerjournal)**: beim dritten gleichen Fehler wechselt der Mechanismus: Prüfung, Regel, Hook
4. **Berechtigungen**: was der Agent ohne Rückfrage darf, was eine Freigabe braucht, welche Daten wohin dürfen





Templates für Context-, Knowledge und Agentic Engineering

- **Promptotyping.** Methode, Skill und Vorlagen für Wissensdokumente. https://github.com/DigitalHumanitiesCraft/Promptotyping
- **Agentic Edition Pipeline.** https://github.com/DigitalHumanitiesCraft/agentic-edition-pipeline
- **Research Mission Control.** Steuerung mehrerer Agenten und Arbeitsstränge. https://github.com/DigitalHumanitiesCraft/research-mission-control
- **Grounded Vault.** Obsidian-Vault als belegpflichtige Wissensbasis für Agenten. https://github.com/DigitalHumanitiesCraft/grounded-vault
- **Second Brain Vault.** Christophers Vault mit Claude Code als Arbeitsbeispiel. https://github.com/chpollin/second-brain-vault

Alle Repositories sind Research Previews. Sie entstehen in laufender Forschungs- und Projektarbeit, ändern sich fortlaufend und zeigen den Stand vom Oktober 2026. Sie dienen dem Nachvollziehen der Methode und als Ausgangspunkt für eigene Experimente, für den produktiven Einsatz sind sie nicht geprüft.